Uniwersytet
Kardynała Stefana Wyszyńskiego
w Warszawie

Remik – projekt programistyczny

Autor: Damian Kantorowski

Spis treści

Wstęp ... 3
Zasady ... 3
Założenia ... 3
Charakterystyka modelu ... 3
Opis programu ... 5
Struktura ... 5
Opis metod ... 6
Schematy ... 10
Kod źródłowy ... 12
Grafika ... 13
Interfejs ... 14
Uruchomienie programu ... 15
Tworzenie pliku wykonywalnego ... 15
Opis elementów gry ... 15
Przebieg rozgrywki ... 15
Bibliografia ... 16
Oświadczenie ... 17

Wstęp
Celem projektu było zrealizowanie w pełni funkcjonalnej gry komputerowej bazującej na karcianej grze Remik, której zasady przedstawiono poniżej.

Zasady
- Każdy z graczy dostaje po 10 kart z klasycznej talii 52 kart + 2 Jokery.
- Celem gry jest pozbycie się wszystkich kart z ręki. Pierwszy gracz, któremu się to uda, wygrywa rundę.
- Na początku tury gracz dobiera kartę ze stosu zakrytego.
- Na zakończenie tury gracz musi odrzucić wybraną ze znajdujących się w jego ręce kart na stos kart odrzuconych.
- Kart można się pozbyć podczas tury na trzy sposoby: wyłożyć na stół, dołożyć do już leżących na stole lub odrzucić na stos.
- Karty mogą być wyłożone na jeden z dwóch sposobów: jako sekwens lub jako grupa.
- Sekwens składa się z trzech lub więcej kart tego samego koloru i ułożonych po kolei, np. 3, 4 i 5 Pik.
- Grupę stanowią trzy lub cztery karty tej samej wartości np. 10 Pik, 10 Trefl i 10 Kier.
- Brakujące karty mogą być zastąpione Jokerami, jednak w danym wyłożeniu nie może być ich więcej niż kart niebędących Jokerami.
- Nie można układać dwóch Jokerów obok siebie.
- Joker znajdujący się na stole może być zamieniony z pasującą kartą.
- Runda kończy się gdy jeden z graczy pozbędzie się wszystkich swoich kart i umieści ostatnią kartę na stosie kart odrzuconych.
- Zwycięzca zdobywa tyle punktów, ile warte są karty pozostałe w ręce przeciwnika.
- Karty od 2 do 10 są liczone według wartości. Walet, Dama i Król są warte 10 pkt., As - 11, Joker - 15.
- Gra kończy się, gdy któryś z graczy zdobędzie co najmniej 100 punktów.

Założenia
Ruchy są walidowane – niemożliwe jest wykonanie ruchu niezgodnego z powyższymi zasadami.

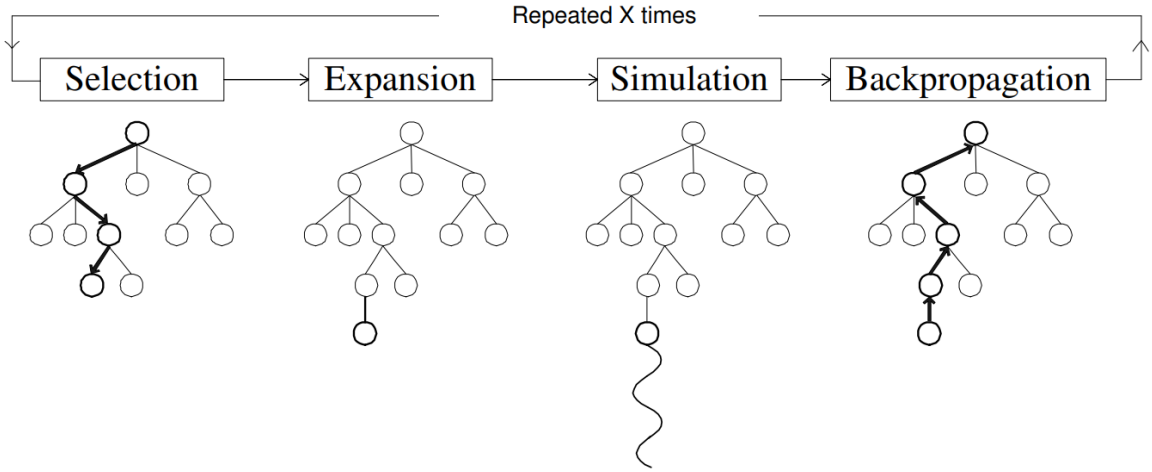
Rozgrywka prowadzona jest pomiędzy dwoma graczami – użytkownikiem programu oraz komputerem wykonującym korzystne dla siebie ruchy.

Gra nie posiada systemu zapisu – każdorazowe uruchomienie programu bądź wyjście do menu oznacza rozpoczęcie rozgrywki od nowa.

Charakterystyka modelu[1]
Komputer wykonuje swoje ruchy z użyciem algorytmu zbudowanego na podstawie przeszukiwania drzew metodą Monte Carlo. Jest to metoda podejmowania decyzji w problemach rozwiązywanych przez zastosowanie sztucznej inteligencji, na przykład w grach. Łączy ona uniwersalność symulacji losowych z precyzją przeszukiwania drzewa. Głównym elementem algorytmu MCTS jest tworzenie drzewa, węzeł po węźle, zgodnie z wynikami przeprowadzonych symulacji rozgrywki. Proces ten składa się z następujących etapów:
1. Wybór (ang. *selection*): zaczynając od korzenia drzewa R wybieraj kolejne węzły potomne, aż dotrzesz do liścia drzewa L (wyjaśnienie poniżej)
2. Rozrost (ang. *expansion*): o ile L nie kończy gry, utwórz w nim jeden lub więcej węzłów potomnych i wybierz z nich jeden węzeł C

3. Symulacja (ang. *simulation*): rozegraj losową symulację z węzła C
4. Propagacja wstecz (ang. *backpropagation*): na podstawie wyniku rozegranej symulacji uaktualnij informacje w węzłach na ścieżce prowadzącej od C do R

Etapy budowania drzewa w algorytmie MCTS[2]:



Wybór węzłów podczas rozwijania drzewa odbywa się poprzez wybranie węzła, który maksymalizuje pewną wartość, analogicznie do „Problemu wielorękiego bandyty", w którym gracz musi wybrać automat maksymalizujący szacunkową nagrodę w każdej turze. Zwykle stosuje się w tym celu wzór górnej granicy ufności dla drzew (UCT) o następującej postaci:

$$\frac{w_i}{n_i} + c\sqrt{\frac{\ln N_i}{n_i}}$$

gdzie
w_i oznacza liczbę wygranych w danym węźle po i-tym ruchu,
n_i oznacza liczbę symulacji w danym węźle po i-tym ruchu,
c to parametr eksploracji – teoretycznie równy $\sqrt{2}$, a w praktyce zwykle dobierany empirycznie,
N_i oznacza liczbę symulacji w rodzicu danego węzła po i-tym ruchu.
Wzór ten pozwala na zachowanie równowagi między eksploatacją ruchów o znanej wygranej a eksploracją rzadziej odwiedzanych węzłów. Węzeł o maksymalnej wartości UCT traktowany jest jako najkorzystniejszy do rozwinięcia. Szacowanie wartości nagrody bazuje na losowych symulacjach, dlatego każdy z węzłów musi zostać odwiedzony wiele razy, zanim te szacunki staną się wiarygodne.
Implementacja algorytmu zastosowana w tym programie posiada jeszcze jeden krok:
0. Determinacja (ang. determinization)

Determinacja wykonywana jest przed pozostałymi krokami i polega na randomizowaniu informacji w korzeniu, które są ukryte przed graczem.
Jako, że przy każdej iteracji algorytmu następuje determinacja, to część stanów będzie różnić się jedynie informacjami zrandomizowanymi. Każdy z węzłów drzewa odpowiada więc nie pojedynczemu stanowi a zestawowi stanów (ang. *information set*), w których gra znajdzie się po wykonaniu wszystkich ruchów ze ścieżki prowadzącej od korzenia do tego węzła. Oznacza to, że węzły znajdujące się na jednym poziomie drzewa zawierają ruchy możliwe do wykonania z ich rodzica. Na przykład, gdy gracz dobrał już kartę ze stosu i karty, które posiada w ręce nie dają mu możliwości wyłożenia lub dołożenia, to możliwymi ruchami z tego stanu będzie odrzucanie każdej z kart z ręki.
Podczas etapu propagacji wstecznej aktualizowana jest liczba odwiedzin oraz wynik danego węzła. Wynik zwiększany jest o 1, jeżeli doprowadził do wygranej komputera, bądź o 0.5, jeżeli doprowadził do remisu. Remis w grze następuje po 20 przetasowaniach talii po jej

wyczerpaniu. Zapobiega to zapętlaniu symulacji w przypadku natrafienia na moment w rundzie niemożliwy do rozstrzygnięcia.
Z uwagi na fakt, iż przy każdym rozwinięciu drzewa wybierany jest węzeł o maksymalnej wartości UCT, często odwiedzane węzły oznaczają korzystne dla komputera ruchy. Po przeprowadzeniu zadanej liczby iteracji algorytmu, wybierany jest ruch o największej liczbie odwiedzin z ruchów możliwych do wykonania z korzenia drzewa.

Opis programu

Środowisko: Microsoft Visual Studio 2022, Python 3.11.3

Program korzysta z zewnętrznej biblioteki graficznej PyGame oraz bibliotek standardowych języka takich jak:
sys, os – dostęp do plików
random – tasowanie talii, losowanie graczy, losowanie ruchów w symulacji
copy – duplikowanie stanu gry na potrzeby symulacji
threading – stworzenie wątku, który w tle znajdzie ruch komputera
itertools – znalezienie kombinacji kart
collections – utworzenie słowników grupujących karty
enum – stworzenie typów wyliczeniowych
math – obliczenia potrzebne do wzoru użytego w UCT

Struktura

W folderze projektu znajdują się następujące pliki:

rummy.py – skrypt główny, zawiera wszystkie elementy interfejsu programu oraz logikę gry.
Znajdują się w nim klasy:
- Card – zawiera kolor, wartość, obraz karty oraz pola pozwalające na ustalenie jej pozycji
- Hand – zawiera listę kart gracza
- Deck – stos zakryty
- Pile – stos odkryty
- Meld – wyłożenie na stole
- Player – zawiera rękę gracza oraz jego wynik
- Button – przycisk sortujący karty
- Game – przechowuje wszystkie informacje o stanie gry

ismcts.py – skrypt zawierający algorytm ISMCTS
Znajdują się w nim klasy:
- Node – węzeł drzewa
- ISMCTS – proces tworzenia i przeszukiwania drzewa

Część kodu z klas Node i ISMCTS została zaczerpnięta z pliku[3] towarzyszącemu artykułowi[4] ze strony aifactory.co.uk, przystosowana do implementacji w grze Remik oraz zaktualizowana do wyższej wersji języka. Zgoda na użycie pomysłu na realizację algorytmu ISMCTS została udzielona przez autorów w komentarzu znajdującym się zarówno w pliku oryginalnym jak i w pliku dołączonym do projektu.

enums.py – zawiera wyliczeniowe typy danych używane w programie
Znajdują się w nim klasy:
- Suits – kolory kart
- Ranks – wartości kart
- Scores – punktacja kart

States – możliwe stany gry
Moves – możliwe ruchy graczy

build.py – skrypt tworzący plik wykonywalny

Opis metod

Klasa Card:

__init__(self, suit, score, front, back) – konstruktor klasy Card

__eq__(self, other) – porównuje dwie karty, sprawdza czy mają ten sam kolor i wartość

__hash__(self) – zwraca hash karty, na potrzeby tworzenia słownikowych struktur danych kart

__repr__(self) – zwraca napisową reprezentację karty

__deepcopy__(self, _) – zwraca nowy obiekt typu Card, kopiując z macierzystego obiektu jedynie pola zawierające kolor i wartość karty

animate(self, destination, speed) – wywoływana jest w każdej klatce dla wszystkich kart, zmienia aktualne położenie karty, dopóki nie znajdzie się ona w pozycji destination

update(self, destination, speed) – aktualizuje pozycje i grafiki kart w każdej klatce

Klasa Hand:

__init__(self) – konstruktor klasy Hand

__deepcopy__(self, _) – zwraca nowy obiekt typu Hand, kopiując wszystkie pola macierzystego obiektu oprócz tych używanych do wyświetlania ręki na ekranie

add_card(self, card) – dodaje kartę do ręki

sort_by_rank(self, card) – sortuje karty w ręce względem wartości, a potem względem koloru

sort_by_suit(self, card) – sortuje karty w ręce względem koloru, a potem po względem wartości

discard(self, card) – usuwa kartę z ręki

calculate_score(self) – zwraca punktację wszystkich kart w ręce

find_card(self, card) – zwraca kartę z ręki, która równa jest karcie podanej w argumencie

find_melds(self) – zapisuje do listy possible_melds wszystkie możliwe wyłożenia, które da się stworzyć z kart znajdujących się w ręce

find_layoffs(self, melds) – zapisuje do listy possible_layoffs wszystkie dołożenia kart w ręce do kart znajdujących się na stole

find_jokers_swaps(self, melds) – zapisuje do listy possible_jokers_swaps wszystkie możliwe zamiany jokerów znajdujących się na stole na karty w ręce

update(self) – aktualizuje pozycje wszystkich kart w ręce

Klasa Deck:

__init__(self, images, jokers_enabled) – konstruktor klasy Deck

__deepcopy__(self, _) – zwraca nowy obiekt typu Deck, kopiując z macierzystego obiektu jedynie listę kart w talii

generate(self, images, jokers_enabled) – tworzy nową talię kart

shuffle(self) – tasuje karty w talii

deal(self) – usuwa ostatnią kartę z talii i zwraca ją

Klasa Pile:

__init__(self) – konstruktor klasy Pile

__deepcopy__(self, _) – zwraca nowy obiekt typu Pile, kopiując z macierzystego obiektu jedynie listę kart na stosie

deal(self) – usuwa ostatnią kartę ze stosu i zwraca ją

put(self, card) – dodaje kartę na stos

update(self) – aktualizuje położenie dwóch ostatnich kart ze stosu

Klasa Meld:

__init__(self) – konstruktor klasy Meld

put_front(self, card) – dodaje kartę na przód wyłożenia

put_back(self, card) – dodaje kartę na tył wyłożenia

deal(self) – usuwa pierwszą możliwą do zabrania kartę ze stosu i zwraca ją

swap_joker(self, card_to_swap) – zamienia Jokera znajdującego się w wyłożeniu z odpowiadającą mu kartą

layoff_possible(self, card) – sprawdza, czy dołożenia danej karty jest możliwe

joker_swap_possible(self, card_to_swap) – sprawdza czy zamiana dowolnego z Jokerów w wyłożeniu z daną kartą jest możliwa

is_valid_run(self) – sprawdza, czy wyłożenie jest prawidłowym sekwensem

is_valid_set(self) – sprawdza, czy wyłożenie jest prawidłową grupą

update(self) – aktualizuje pozycje wszystkich kart w danych wyłożeniu

Klasa Player:

__init__(self) – konstruktor klasy Player

draw_deck(self, deck) – dobiera kartę ze stosu zakrytego

draw_pile(self, deck) – dobiera kartę ze stosu odkrytego bądź nieukończonego wyłożenia

select_card(self, mouse_pos) – po wciśnięciu lewego przycisku myszy zapisuje w polu selected_card kartę z ręki, która znajduje się pod kursorem

discard_card(self, card, pile) – przenosi kartę z ręki na stos odkryty

add_to_meld(self, meld, card, back=False) – dodaje kartę do wyłożenia

swap_joker(self, meld, card) – zamienia Jokera w wyłożeniu na odpowiadającą mu kartę

sort_hand(self) – sortuje karty w ręce gracza po naciśnięciu przycisku na ekranie

move_card(self, mouse_pos, game_state, is_players_turn) – zmienia pozycję karty po wybraniu jej I przesunięciu kursora myszy

Klasa Button:

__init__(self) – konstruktor klasy Button

update(self) – zmienia pozycję przycisku po jego naciśnięciu

Klasa Game:

__init__(self) – konstruktor klasy Game

load_images(self) – zwraca grafiki załadowane z folderu images znajdującego się w folderze projektu, bądź w folderze tymczasowym w przypadku uruchamiania programu z pliku wykonywalnego

deal_cards(self) – rozdaje karty graczom na początku rundy

get_computers_move(self) – w przypadku, gdy nastąpi kolej komputera by wykonać ruch, uruchamia proces tworzenia i przeszukiwania drzewa oraz wykonuje znaleziony najlepszy ruch

validate_melds(self) – sprawdza, czy wszystkie wyłożenia znajdujące się na stole są poprawnymi sekwencjami albo grupami

set_placeholders(self) – ustawia odpowiednią pozycję wyłożeń oraz ostatniego, wolnego pola

pile_to_deck(self) – po przetasowaniu, dokonuje przełożenia kart ze stosu odkrytego na stos zakryty w przypadku, gdy karty na stosie zakrytym się wyczerpały

fix_card(self) – podnosi flagę fixed dla wszystkich kart znajdujących się w wyłożeniach, tak aby nie dało się ich zabrać z powrotem do ręki

clone_and_randomize(self) – zwraca nowy obiekt typu Game, posiadający skopiowane z macierzystego obiektu tylko pola konieczne do przeprowadzenia symulacji. Następnie „ukrywa” karty, których komputer do tej pory nie poznał tzn., że karty o których nie ma informacji wybierane są z listy potasowanych kart znajdujących się na stosie zakrytym

get_moves(self) – zwraca listę ruchów legalnych z danego stanu gry

do_move(self, move) – wykonuje ruch podany w argumencie i aktualizuje stan gry

progres_state(self) – dokonuje przejścia do kolejnego stanu gry oraz zmiany gracza, którego ruch będzie następny

is_players_turn(self) – sprawdza, czy aktualna tura należy do gracza

get_result(self, player) – zwraca wynik zakończonej gry z perspektywy gracza podanego w argumencie, metoda jest używana w propagacji wstecznej po przeprowadzeniu symulacji

is_game_over(self) – sprawdza, czy któryś z graczy osiągnął 100 punktów

check_winners(self) – po skończonej grze aktualizuje punkty wygranego

restart_round(self) – tworzy nową rundę, przywraca większość pól klasy Game do wartości domyślnych, zachowując aktualne punkty graczy

change_resolution(self) – przełącza rozmiar okna po wybraniu go w menu

draw_menu(self) – wyświetla widok menu na ekranie

draw_leaderboard(self) – wyświetla widok zakończonej rundy na ekranie

draw_text(self, text, pos, color) – pozwala na wygenerowanie i wyświetlenie tekstu z domyślną dla systemu czcionką w pozycji pos

handle_event(self, event) – metoda obsługująca zdarzenia wykryte w oknie gry takie jak wykonanie ruchu przez gracza

update(self) – w każdej klatce aktualizuje pozycje wszystkich elementów na ekranie

draw(self) – w każdej klatce wyświetla odpowiednią zawartość ekranu w zależności od stanu gry

Klasa Node:

__init__(self, move, parent, player) – konstruktor klasy Node

get_untried_moves(self, legal_moves) – zwraca listę ruchów, które nie znajdują się wśród dzieci węzła

select_child(self, legal_moves, exploration) – używa wzoru UCT by znaleźć dziecko, którego gałąź powinna zostać rozwinięta

add_child(self, move, player) – dodaje nowe dziecko do węzła i je zwraca

update(self, terminal_state) – aktualizuje liczbę odwiedzin oraz zwycięstw w danym węźle

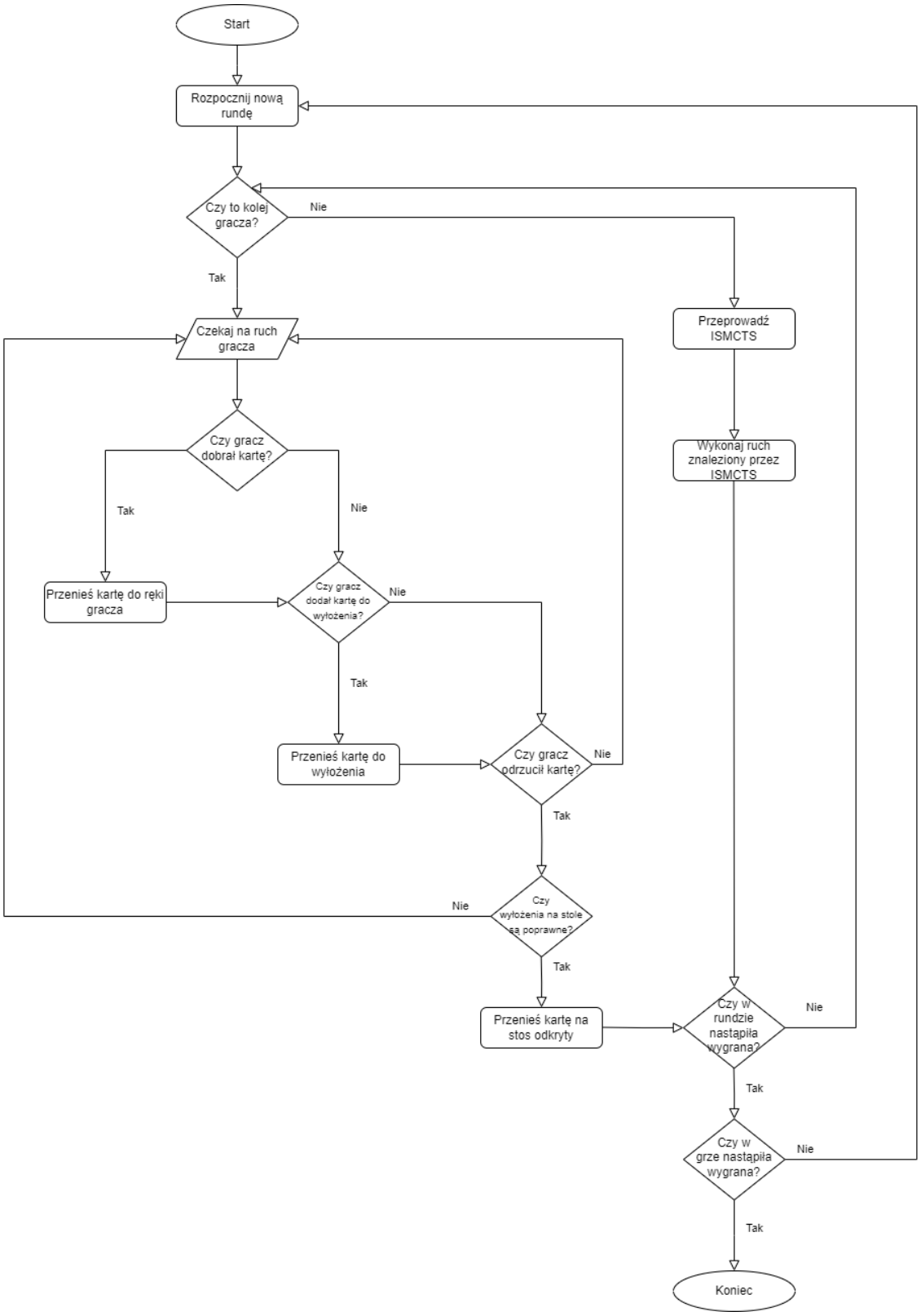
Klasa ISMCTS:

__init__(self) – konstruktor klasy ISMCTS

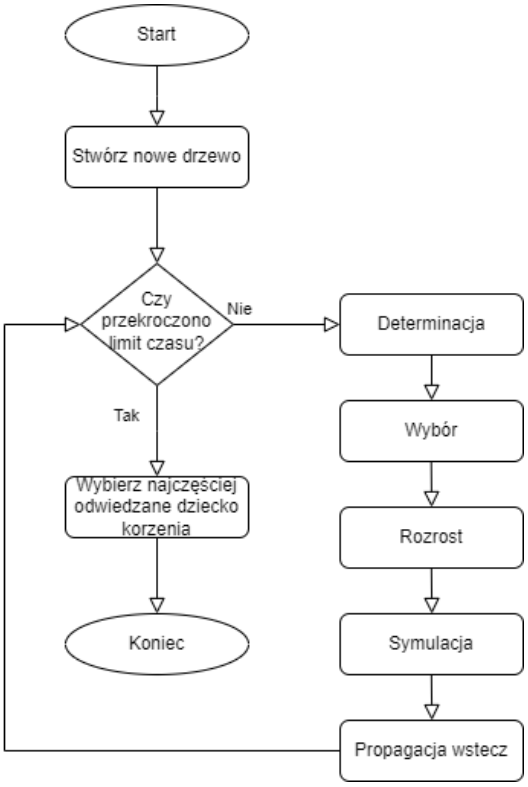
run(self, root_state, timeout=2) – wykonuje przeszukiwanie drzewa dla stanu gry root_state w celu znalezienia najlepszego ruchu, który zostaje zapisane w polu best_move

Schematy

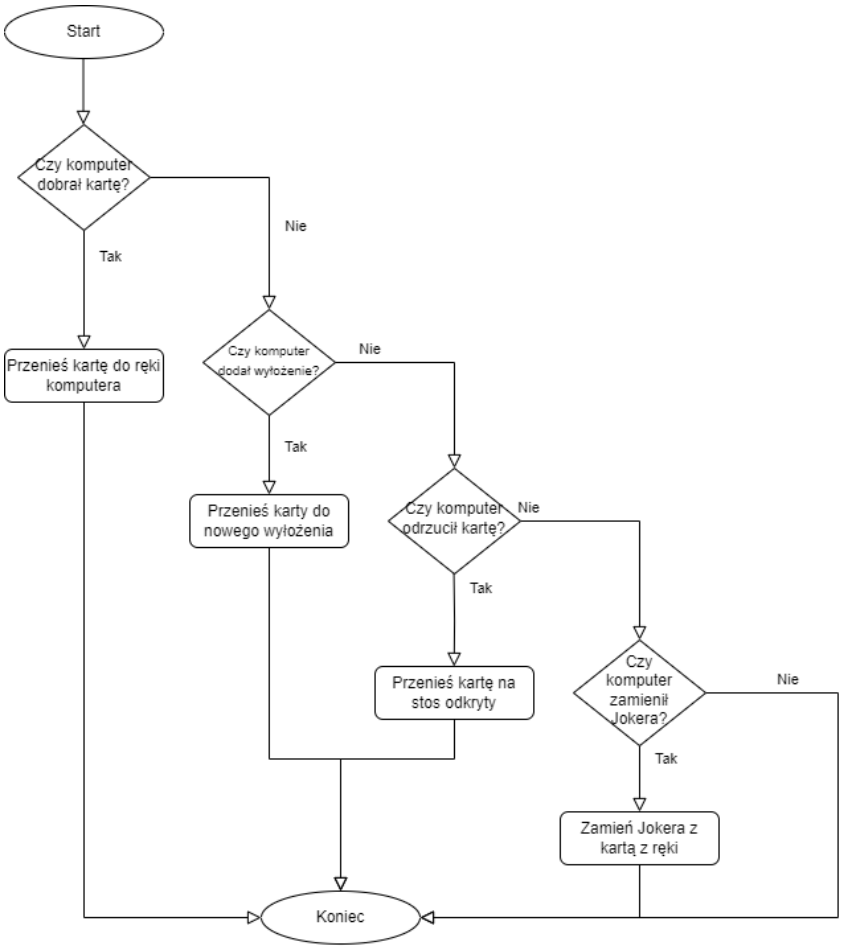
Schemat pojedynczej rundy Remika:



Schemat algorytmu ISMCTS:



Schemat wykonywania ruchów komputera:



Kod źródłowy

Najistotniejsze fragmenty kodu źródłowego to:

Wyszukiwanie możliwych wyłożeń – posortuj listę kart, pogrupuj karty po wartościach oraz po kolorach, przejdź przez karty pogrupowane po kolorach w poszukiwaniu sekwensów. Następnie przejdź przez karty pogrupowane po wartościach w poszukiwaniu grup. Metoda znajduje wszystkie możliwe kombinacje w tym takie, które uzupełnione są Jokerem.

```
def find_melds(self):
    self.possible_melds = []
    ranks = defaultdict(set)
    suits = defaultdict(list)
    self.sort_by_suit()
    for card in self.cards:
        ranks[card.rank].add(card)
        suits[card.suit].append(card)
    joker = None
    if Ranks.JOKER in ranks:
        joker = ranks[Ranks.JOKER].pop()
        del suits[4]
    for _, cards in suits.items():
        for i, first_card in enumerate(cards):
            for j, last_card in enumerate(cards[i+1:]):
                if first_card.rank + j + 1 == last_card.rank:
                    if joker and j > 0:
                        self.possible_melds.append([joker] + cards[i:i+j+1])
                        self.possible_melds.append(cards[i:i+j+1] + [joker])
                    if j > 1:
                        self.possible_melds.append(cards[i:i+j+1])
                elif joker and first_card.rank + j + 2 == last_card.rank:
                    for k, card in enumerate(cards[i+1:]):
                        if card.rank != first_card.rank + 1:
                            break
                    self.possible_melds.append(cards[i:i+k+1] + [joker] + cards[i+k+1:i+j+2])
    for _, cards in ranks.items():
        if len(cards) >= 3:
            for meld in combinations(cards, 3):
                self.possible_melds.append(meld)
                if joker:
                    self.possible_melds.append(list(meld) + [joker])
            if len(cards) == 4:
                self.possible_melds.append(cards)
```

Metoda, która pozyskuje i wykonuje ruch komputera. Każde wyszukiwanie ruchu odbywa się na osobnym wątku, dzięki czemu interfejs programu pozostaje responsywny.

```
def get_computers_move(self):
    if not self.is_players_turn() and self.state in [
        States.DRAW, States.MELD, States.LAY_OFF, States.DISCARD]:
        if self.search.best_move is not None:
            self.do_move(self.search.best_move)
            self.search.best_move = None
            self.fix_cards()
        elif self.get_moves() == [Moves.PASS]:
            self.do_move(Moves.PASS)
        elif self.search.thread is None or not self.search.thread.is_alive():
            self.search.thread = Thread(
                target = self.search.run,
                args = [self],
                daemon = True)
            self.search.thread.start()
```

Metody dostarczane przez klasę Node pozwalające na przeprowadzenie wyszukiwania w drzewie.

```
def get_untried_moves(self, legal_moves):
    tried_moves = [child.move for child in self.children]
    return [move for move in legal_moves if move not in tried_moves]

def select_child(self, legal_moves, exploration=0.7):
    legal_children = [child for child in self.children if child.move in legal_moves]
    for child in legal_children:
        child.considerations += 1
    return max(legal_children,
               key=lambda c: float(c.wins) / float(c.visits)
               + exploration * sqrt(log(c.considerations) / float(c.visits)))

def add_child(self, move, player):
    new = Node(move, self, player)
    self.children.append(new)
    return new

def update(self, terminal_state):
    self.visits += 1
    if self.player is not None:
        self.wins += terminal_state.get_result(self.player)
```

Metoda run klasy ISMCTS przeprowadzająca wyszukiwanie najlepszego ruchu z danego stanu gry. Kończy działanie po czasie określonym w parametrze timeout.

```
def run(self, root_state, timeout=2):
    start = time()
    root_node = Node()
    while time() < start + timeout:
        node = root_node
        state = root_state.clone_and_randomize()
        while True:
            #Select
            while (legal_moves := state.get_moves()) and not node.get_untried_moves(legal_moves):
                node = node.select_child(legal_moves)
                state.do_move(node.move)
            #Expand
            if (legal_moves := state.get_moves()) and (untried_moves := node.get_untried_moves(legal_moves)):
                if legal_moves == [Moves.PASS]:
                    state.do_move(Moves.PASS)
                    continue
                move = choice(untried_moves)
                player = state.current_player
                node = node.add_child(move, player)
                state.do_move(move)
            #Simulate
            while legal_moves := state.get_moves():
                state.do_move(choice(legal_moves))
            #Backpropagate
            while node is not None:
                node.update(state)
                node = node.parent
            break
    self.best_move = max(root_node.children, key=lambda c: c.visits).move
```

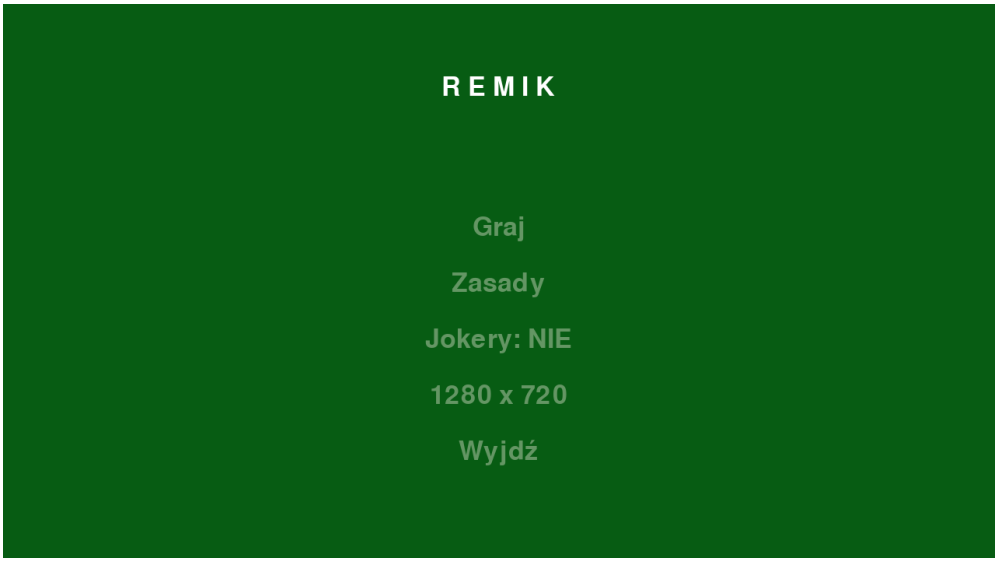
Grafika

Grafiki kart[5] są udostępnione na licencji otwartej. Ich autorem jest Byron Knoll.

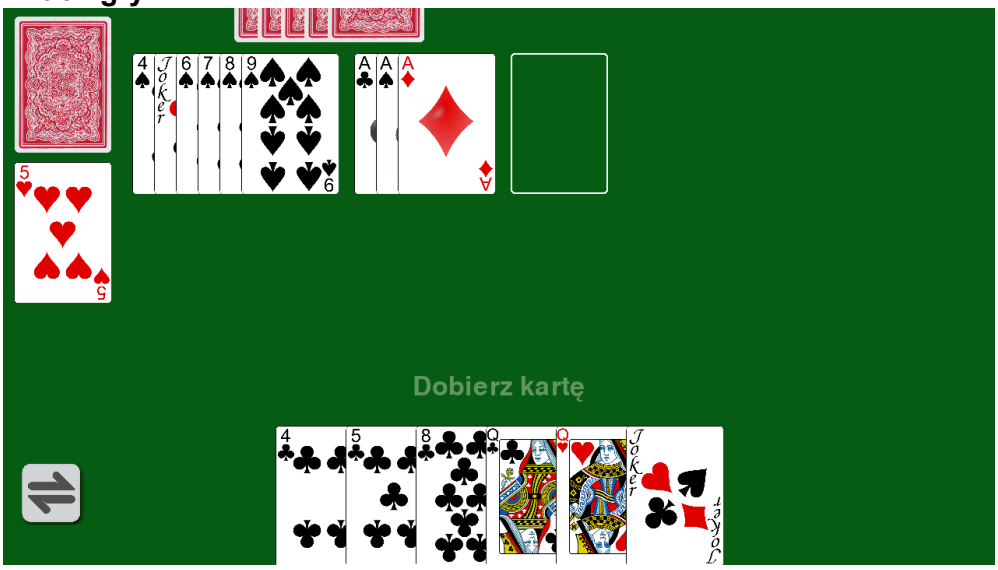
Wszystkie elementy interfejsu zostały stworzone z użyciem biblioteki PyGame.

Interfejs

Widok menu:



Widok gry:



Widok końca rundy:

Uruchomienie programu

Wymagania: Python >=3.8, PyGame

```
$ pip install pygame
$ py rummy.py
```

Plik wykonywalny można uruchomić bez instalacji Pythona i zależności.

Tworzenie pliku wykonywalnego

Wymagania: Python >=3.8, PyGame, PyInstaller

```
$ pip install pygame pyinstaller
$ py build.py
```

Opis elementów gry

Po uruchomieniu gry wyświetla się widok menu. Można do niego powrócić w dowolnym momencie poprzez naciśniecie klawisza Esc na klawiaturze.
Menu zawiera pięć opcji:

- Graj – uruchamia grę
- Zasady – otwiera plik tekstowy z zasadami
- Jokery: TAK / Jokery: NIE – pozwala na dodanie lub usuniecie Jokerów
- 1280 x 720 / 1920 x 1080 – zmienia rozmiar okna
- Wyjdź – zamyka program

W widoku gry na ekranie znajdują się następujące elementy:

- Stos zakryty – lewy górny róg
- Stos odkryty – poniżej stosu zakrytego
- Karty w ręce komputera – góra ekranu
- Karty w ręce gracza – dół ekranu
- Karty na stole – pomiędzy ręką komputera a ręką gracza
- Przycisk sortujący karty w ręce gracza – lewy dolny róg. Jego naciśnięcie powoduje sortowanie kart naprzemiennie według kolorów oraz wartości

W widoku zakończonej rundy na ekranie znajdują się następujące elementy:

- Wartość kart znajdujących się w ręce przegranego
- Karty znajdujące się w ręce przegranego
- Aktualny wynik obydwóch graczy
- Napis „Kolejne rozdanie”, którego naciśnięcie uruchamia następną rundę

Przebieg rozgrywki

Po wybraniu opcji Graj program przechodzi do widoku gry. Następuje utworzenie talii, jej przetasowanie, rozdanie początkowych kart graczom i wyłożenie ostatniej karty z talii na stos odkryty. Losowany jest rozpoczynający gracz. Jeżeli rozpoczyna komputer, nad ręką gracza wyświetli się komunikat „Komputer myśli…”, w przeciwnym wypadku będzie to komunikat „Dobierz kartę”. Wtedy gracz decyduje, czy dobrać kartę ze stosu zakrytego czy odkrytego. Potwierdza wybór poprzez kliknięcie na odpowiedni stos. Jeżeli po tym ruchu w ręce gracza znajdują się karty tworzące zgodne z zasadami wyłożenie, może on przenieść je na stół.

Dokonać tego należy przeciągając karty w odpowiedniej kolejności, począwszy od umieszczenia pierwszej karty na wolnym polu z białą ramką i dokładanie następnych kart. Jeżeli pośród kart gracza nie ma poprawnych wyłożeń bądź też podejmie on decyzje o zatrzymaniu wyłożeń na późniejszy etap gry, kolejnym krokiem jest odrzucenie wybranej karty z ręki. Kartę należy przeciągnąć i upuścić nad stos odkryty. Wykonanie tego ruchu kończy turę i następuje kolej przeciwnika. Jeżeli na stole znalazły się karty nietworzące poprawnego wyłożenia zostanie wyświetlony komunikat „Nieprawidłowe ułożenie kart", a gracz będzie miał szasnę na poprawienie błędu przez zabranie kart ze stołu z powrotem do ręki. Gdy na stole znajdą się wyłożenia możliwe do uzupełnienia kartami z ręki, gracz może wykonać dołożenie. Gdy na stole znajdzie się wyłożenie z Jokerem, gracz może zamienić go na odpowiadającą mu kartę upuszczając ją nad tym wyłożeniem. Liczba wyłożeń i dołożeń w jednej turze jest dowolna, to znaczy, że są one opcjonalne i nie powodują zakończenia tury. Gracze wykonują ruchy na przemian, dopóki któryś z nich nie pozbędzie się wszystkich kart. Warunkiem wygranej jest położenie na stosie odkrytym ostatniej karty z ręki, co oznacza, że wyłożenie ostatniej karty na stół nie jest ruchem wygrywającym. Na koniec rundy wyświetla się podsumowanie a gracz dostaje możliwość przejścia do kolejnej, o ile nie osiągnięto jeszcze pułapu 100 punktów. Jeżeli któryś z graczy tego dokona, to następuje koniec gry i nie będzie możliwości kontynuowania jej w następnej rundzie.

Bibliografia

[1] https://pl.wikipedia.org/wiki/Monte-Carlo_Tree_Search

[2] F. van Lishout, G. Chaslot, J. Uiterwijk, *Monte-Carlo Tree Search in Backgammon* https://www.researchgate.net/publication/228378473_Monte-Carlo_tree_search_in_backgammon

[3] E. Powley, D. Whitehouse, P. Cowling, *ISMCTS.txt* http://www.aifactory.co.uk/newsletter/ISMCTS.txt

[4] E. Powley, D. Whitehouse, P. Cowling, *Reducing the burden of knowledge: Simulation-based methods in imperfect information games* https://www.aifactory.co.uk/newsletter/2013_01_reduce_burden.htm

[5] B. Knoll, *Vector Playing Cards* http://byronknoll.blogspot.com/2011/03/vector-playing-cards.html

Damian Kantorowski

Nr albumu 119226

Informatyka II rok, I stopień, stacjonarne

Projekt programistyczny indywidualny

Oświadczenie

1. Świadomy odpowiedzialności prawnej oświadczam, że przesłana praca pisemna pod nazwą „Remik – projekt programistyczny” została napisana przeze mnie samodzielnie i nie zawiera treści uzyskanych w sposób niezgodny z obowiązującymi przepisami.

2. Oświadczam, że jestem autorem przesłanej pracy i przysługują mi w zakresie objętym tym oświadczeniem osobiste prawa autorskie do tej pracy.

Damian Kantorowski